

Project 2: Coronary Disease Prediction

Baudart Alexandre

Last update: September 23, 2026

Contents

1	Dataset and Exploratory Analysis	3
1.1	Dataset overview	3
1.2	About the target	3
1.3	Features Description	4
1.4	Excluded features	6
1.5	Missing values	6
1.6	Correlation Analysis	7
2	Data splitting and Preprocessing	9
2.1	Data splitting	9
2.2	Imputing missing values	9
2.3	Preprocessing Pipeline	10
3	Modeling	11
3.1	Models	11
3.2	Hyperparameter Optimization	11
3.3	Training	14
4	Results and discussion	15
4.1	Metrics evaluation	15
4.2	Confusion matrices analysis	16
5	Features importance	16
6	Calibration	17
6.1	Calibration metrics evaluation	17
6.2	Confusion matrices analysis	18
7	Model rankings	19
8	Conclusion	21
	References	21

1 Dataset and Exploratory Analysis

1.1 Dataset overview

The dataset **UCI Heart Disease** has been used for this project [1]. It contains **920 examples** including information related to the patients cardiovascular health such as blood pressure, cholesterol levels, fasting blood sugar and so on.

Originally, this dataset had 76 features but the commonly used version is a subset of those features containing **14 features**. That is this version that has been used for this project. It includes exactly **13 features and one target**, indicating the presence level of a coronary disease on a scale of 0 to 4. It is important to notice that **this dataset is specifically about coronary diseases** (affecting coronary arteries) rather than the whole cardiovascular pathologies.

This dataset is also particular in view of its construction, because it contains information coming from **4 different databases**: the Cleveland Clinic (Ohio, USA), the Hungarian Institutes of Cardiology in Budapest (Hungary), the Veterans Administration Medical Center in Long Beach (California, USA) and the University Hospitals of Zurich and Basel (Switzerland).

1.2 About the target

As mentioned previously, the target *num* shows the disease severity level with the patient. It takes a value between 0 and 4, where 0 represents the absense of coronary disease whereas the superior values indicate higher disease severity levels.

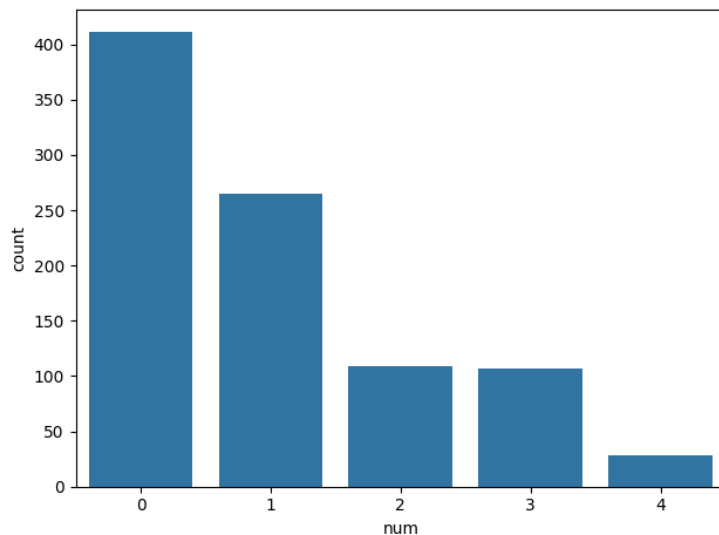


Figure 1: Distribution of the target variable *num*

As we can see above, the dataset includes a higher proportion of patients presentinf with no coronary disease: 420 patients or thereabouts, that represents almost half of the dataset. As the disease level increases, the number of examples decreases. The highest presence level of the disease (4) concernes only 20 or so patients.

This distribution highlights an important **class unbalancing**. An approach to limit this problem was to transform this multiclass problem to a **binary problem**. By doing that, the patients were splitted into two categories: *ill* (0) and *not ill* (1, 2, 3, 4).

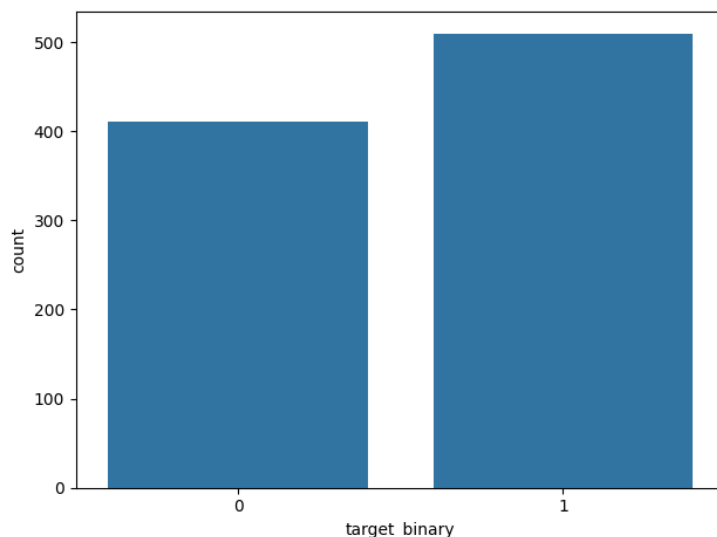


Figure 2: Distribution of the new binarized target variable: *target_binary*

This transformation resulted in a more **balanced class distribution**. The positive class, corresponding to the patients presenting with a disease severity level 1 or superior, even ended up slightly outnumbering the negative class.

This binarization generated of course a loss of information about the the disease severity, but the goal of that project was to detect the presence of a coronary disease. Therefore, this transformation allowed us to express this problem as a binary classification task, while getting a class distribution far more balanced.

1.3 Features Description

The dataset includes 14 variables, that can be distinguished according to their type. Variables are divided into **numerical** and **categorical** types, the latter being subdivided into **nominal** and **ordinal** variables.

Numerical Variables:

Feature	Meaning
<i>age</i>	Patient age
<i>chol</i>	Serum cholesterol
<i>oldpeak</i>	ST depression induced by exercise
<i>trestbps</i>	Resting blood pressure
<i>thalch</i>	Maximum heart rate

Table 1: Numerical variables

CATEGORICAL VARIABLES

As mentioned earlier, the categorical variables can be divided into 2 sub-categories: the nominal variables, with no natural order in the categories and the ordinal variables, for which the categories can be naturally ordered.

Feature	Meaning	Values
<i>sex</i>	Patient sex	Male / Female
<i>cp</i>	Chest pain type	typical angina / atypical angina / non-anginal / asymptomatic
<i>fbs</i>	Fasting blood pressure ≥ 120 mg	True / False
<i>restecg</i>	Resting electrocardiogram results	normal / st-t wave abnormality / left ventricular hypertrophy
<i>exang</i>	Exercise-induced angina	True / False
<i>ca</i>	Number of major vessels colored by fluoroscopy	0 / 1 / 2 / 3
<i>thal</i>	Thallium stress test result	normal / fixed defect / reversible defect

Table 2: Nominal categorical variables

Note: The variable *restecg* described the result of the electrocardiogram at rest. The three categories corresponding respectively to a normal result, a ST-T abnormality and a left ventricular hypertrophy.

Feature	Meaning	Values
<i>slope</i>	ST segment slope at peak exercise	upsloping / flat / downsloping

Table 3: Ordinal categorical variables

1.4 Excluded features

The original dataset also contains two additional columns, *id* and *dataset*. These variables were excluded from the analysis and from the subsequent modelling process, as they do not contain clinical information relevant to the detection of coronary heart disease. The *id* variable is only an identifier for each patient, while *dataset* indicates the database from which the observation originates.

Including these variables could introduce information unrelated to the patient's clinical condition and, in the case of *id*, would not provide a meaningful signal for predicting the target. Therefore, both features were removed before the preprocessing and modelling steps.

1.5 Missing values

There are the missing values number for all the variables:

Feature	Missing values
<i>ca</i>	611
<i>thal</i>	486
<i>slope</i>	309
<i>fbs</i>	90
<i>oldpeak</i>	60
<i>trestbps</i>	59
<i>exang</i>	55
<i>thalch</i>	55
<i>chol</i>	30
<i>restecg</i>	2
<i>cp</i>	0
<i>age</i>	0
<i>sex</i>	0
<i>target_binary</i>	0

Table 4: Missing values

As showing by the table above, the missing values are particularly concentrated on the variables *ca*, *thal* et *slope*. The target variable *target_binary* does not contains any missing values, that allows to keep all the observations for which a class is available. The different strategies used to treat the missing values will be detailed in the section dedicated to the [preprocessing](#).

1.6 Correlation Analysis

An analysis of the matrices built from the **Pearson correlation coefficients** was produced in order to quantify the linear associations between explanatory variables represented numerically.

First of all, by taking into account the target variable:

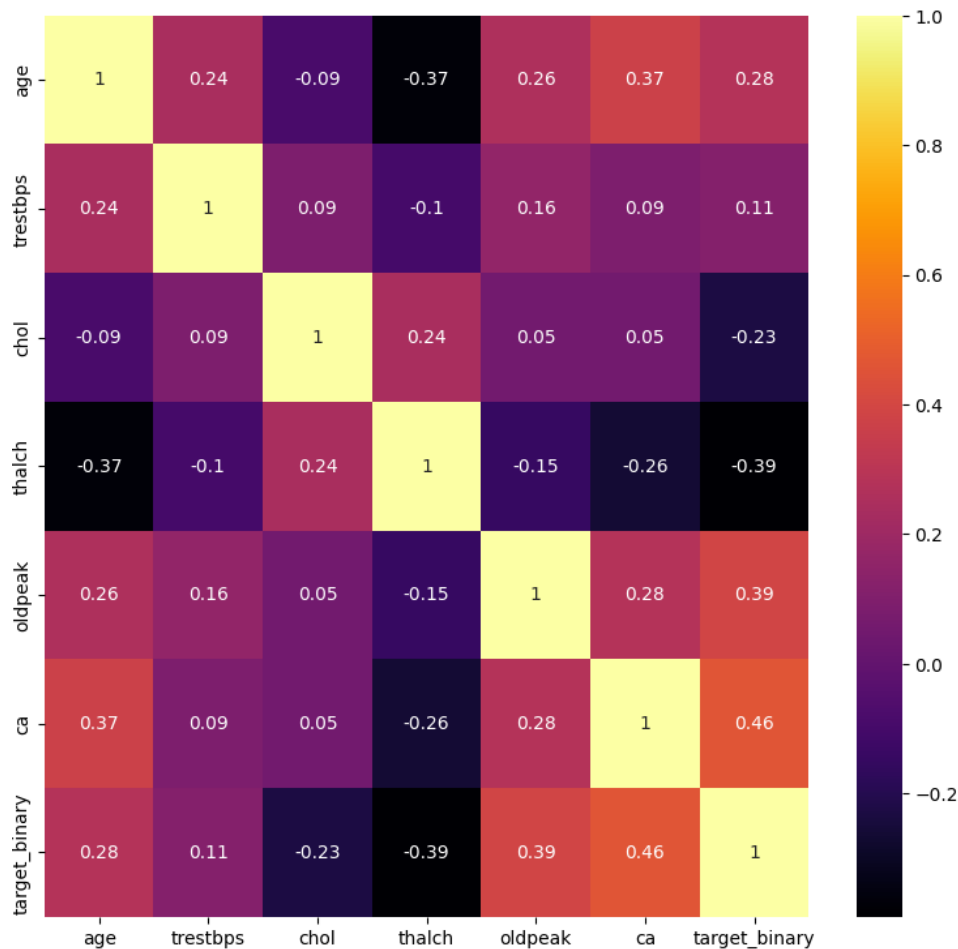


Figure 3: Correlation matrix with target

We can observe that variables such as *oldpeak* and *ca* present a moderated positive association with *target_binary*, whereas *thalch* has a moderated negative association with this latter. The patients presenting with a coronary disease have a tendency, in this sample, to have higher values of *oldpeak* and *ca*, along with lower values of *thalch*

These correlations describe associations between variables and the target, however, they don't allow to establish any causal relationship.

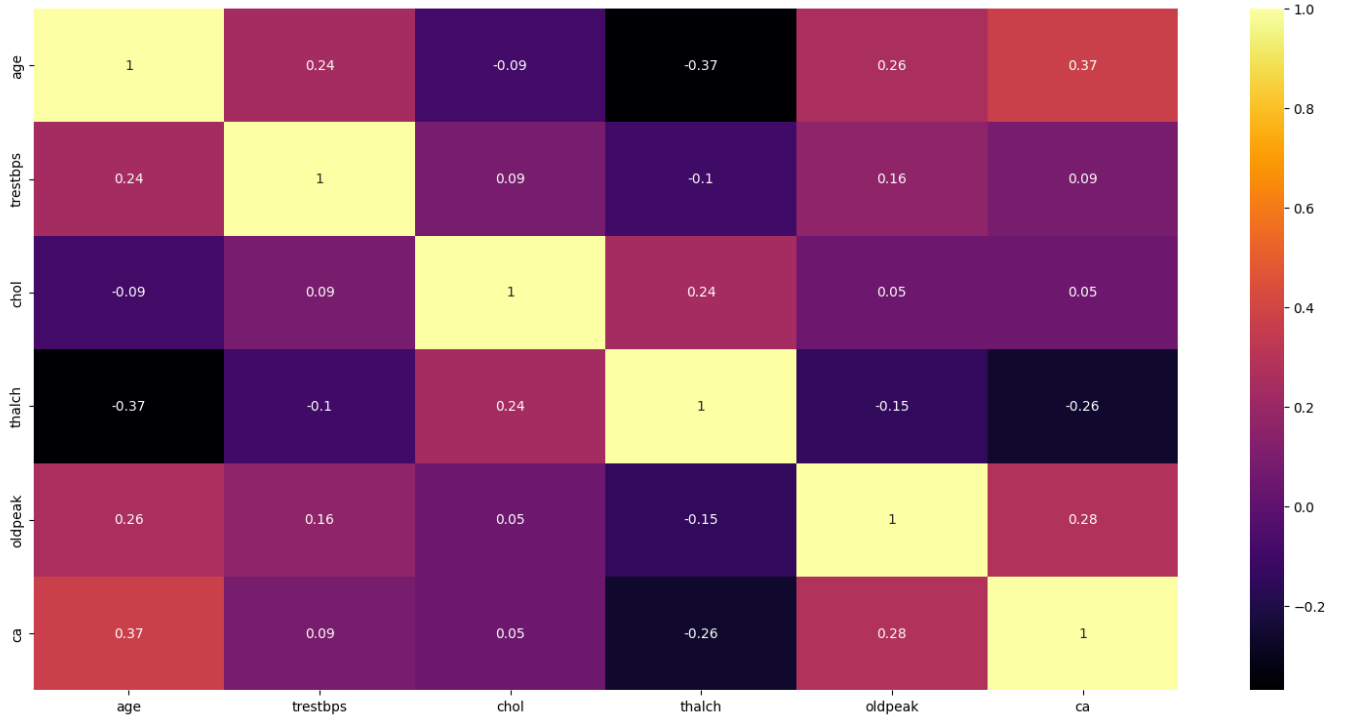


Figure 4: Correlation matrix without target

By removing the target variable from the analysis, we can observe the potential associations between the explanatory variables. We particularly notice that there are moderated positive correlations between *oldpeak* and *ca*, as well as between *ca* and *age*. Some negative correlations also exist, especially between *age* and *thalch*, plus between *thalch* and *ca*.

Thereby, some variables show associations that can reflect some common aspects related to the age and the physiological response to effort. For instance, *age* and *thalch* present a negative correlation, indicating that the oldest patients from this sample tend to get a lower maximum heart rate.

2 Data splitting and Preprocessing

2.1 Data splitting

Before taking the preprocessing up, it is necessary to present how the initial dataset was splitted between the different phases of the project. The dataset was firstly divided into training and test sets using a **80/20 stratified split**. Then, the training set was splitted into the final training set and a calibration set (90/10).

This initial split particularly mitigate the risks regarding **data leakage**, by isolating from the beginning the test set from the different train, validation and optimization steps. Moreover, it organizes more efficiently the data loading in memory: for example, the test set can remain unused during the steps requiring only the train set, such as cross validation, hyperparameters optimization or training.

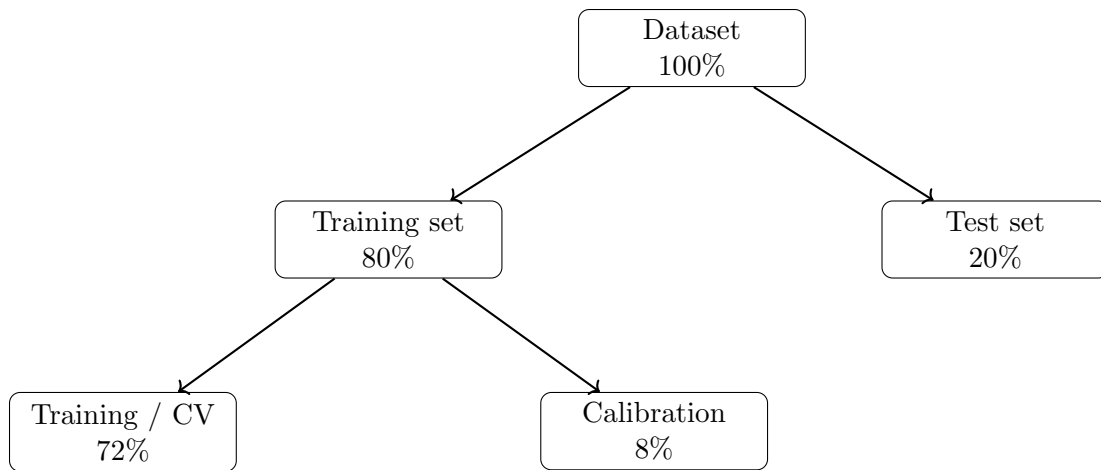


Figure 5: Data splitting strategy

2.2 Imputing missing values

In order to treat the missing values, several imputation methods were used according to the variable type. The choice of the method allow particularly to preserve the categorical nature of the variables while keeping, when it's relevant, the information indicating that a value was initially missing.

Feature type	Imputation method
Numerical	Mediane (+ missing flag – only for variables with 5% of missing values)
Nominal categorical	<i>Unknown</i>
Nominal binary categorical	Mode
Ordinal categorical	Mode (+ missing flag)

Table 5: Preprocessing of the missing values

For numerical variables, median was used to limit the potential influence of outliers, while an

additional flag allow to conserve the information regarding the initial missing of the value. For the nominal variables, a category ***Unknown*** was created to represent explicitly the missing values. As for the binary categorical variables, they were imputed with the mode. Finally, the ordinal variables were also imputed with their mode, while a flag identify the observations for which the value were initially missing.

Note: *cp* et *ca* are nominal variables, but they were treated with the imputation method used for the ordinal variables, that is imputation by the mode + missing flag.

2.3 Preprocessing Pipeline

A preprocessing pipeline was set up in order to gather all the preprocessing steps within a common object. This allow to apply in a consitent and reproducible way the same transformations to data during the different phases of the project. It's pertinent to note that **this pipeline is adjustable and suitable for each model/project**.

To avoid any **data leakage**, the pipeline must be fitted only on the training data. This is valid as well for the hyperparameters optimization as for the final training or the different iterations of the cross-validation. The fitted transformations are then applied on the validation or test data without being readjusted on them.

PIPELINE COMPONENTS

Component	Task	Models
MissingValuesTransformer	Imputation of the missing values according to the methods exposed above	All
BinaryFeaturesTransformer	Encodes the binary variables into integer (0 / 1)	All
ToStringTransformer	Converts the variable <i>ca</i> into a categorical variable to avoid its normalization by the scaling step	All
Scaling	Applies standardization to the numerical variables	Logistic Regression, SVM, MLP and Tab-Transformer
CategoricalEncoder	Encodes the categorical variable into a one-hot vector	All except CatBoost

Table 6: Preprocessing pipeline components

3 Modeling

3.1 Models

Note: As for now, variables will be called features.

The following models were evaluated in this project:

Model	Type
Logistic Regression (baseline)	Linear
SVM	Kernel
Random Forest	Ensemble
XGBoost	Boosting
CatBoost	Boosting
Multi-Layer-Perceptron	Neural network
Tab-Transformer	Neural network

Table 7: Models used in this project

Note: For *Tab-Transformer*, two architecture versions were designed. The first encodes only the categorical features, beforehand represented in the form of one-hot vectors, into embeddings. As for the numerical features, they are treated separately, and concatenated with the representation coming from the attention mechanism applied to the categorical embeddings. The concatenation is then transmitted to the final MLP.

The second version adds the encoding of the numerical variables into embeddings. The representation obtained from the attention mechanism applied to both numerical and categorical embeddings are then concatenated before the transmission to the FFN.

3.2 Hyperparameter Optimization

The optimization of the models hyperparameters was realized using two different tools:

- **Optuna** for classical Machine Learning models.
- **Ray Tune** for Deep Learning models (MLP and Transformer).

In order to limit the computational cost of this step, a **sub-sample equivalent to 30% of the training set** was used during this optimization. This sub-sample was not removed from the training set and it's only used to accelerate the research of the best configurations. Once those configurations determined, the models were trained with their best hyperparameters on the full training set.

The best configuration for each model are presented below:

LOGISTIC REGRESSION

Parameter	Value
C	0.74
tol	2e-4
penalty	L2
max_iter	1000
fit_intercept	True

Table 8: Main parameters for logistic regression

SVM

Parameter	Value
kernel	rbf
C	4.19
gamma	0.0329

Table 9: Main parameters for SVM

RANDOM FOREST

Parameter	Value
n_estimators	307
max_depth	9
min_samples_split	3
min_samples_leaf	2
max_samples	0.70

Table 10: Main parameters for random forest

XGBoost

Parameter	Value
n_estimators	900
learning_rate	0.09
max_depth	6
min_child_weight	2
subsample	0.8
colsample_bytree	0.84
reg_alpha	0.11
reg_lambda	0.01

Table 11: Main parameters for XGBoost

CATBOOST

Parameter	Value
n_estimators	1881
learning_rate	0.07
depth	4
l2_leaf_reg	0.02
cat_features	All the categorical variables

Table 12: Main parameters for CatBoost

MLP

Parameter	Value
n_layers	4
hidden_size	128
dropout	0.13
weight_decay	1e-3 (coeff weight-decay optimizer)

Table 13: Main parameters for mlp

TAB-TRANSFORMER (V1)

Parameter	Value
n_heads (attention)	4
dim (embedding)	32
n_layers (FFN)	2
hidden_size (FFN)	64
dropout (FFN)	0.13
weight_decay	0.015 (coeff weight-decay optimizer)

Table 14: Main parameters for Tab-Transformer-V1

TAB-TRANSFORMER (V2)

Parameter	Value
emb_mlp_hidden_size	64
n_heads (attention)	8
dim (embedding)	32
n_layers (FFN)	2
hidden_size (FFN)	128
dropout (FFN)	0.18
weight_decay	1e-2 (coeff weight-decay optimizer)

Table 15: Main parameters for Tab-Transformer-V2

Note: `emb_mlp_hidden_size` is the `hidden_size` of the embedding MLP in charge of numerical variables.

3.3 Training

After the hyperparameters optimization, the classical models were evaluated by a **stratified 5 splits cross-validation** on the train dataset. XGBoost and CatBoost used also an internal validation dataset (representing 20% of training data) to monitor the performances over iterations and apply an *early stopping* based on ROC-AUC and a patience of 50 iterations. Final models are then trained separately on the full training dataset one last time.

Model	Mean CV ROC-AUC	Std CV ROC-AUC
Logistic Regression	0.8921	0.0353
SVM	0.8997	0.0276
Random Forest	0.8964	0.0292
XGBoost	0.8964	0.0292
CatBoost	0.9022	0.0288

Table 16: Cross-validation results

The performances are close and relatively stable over the different splits. CatBoost gets the best Mean ROC-AUC while SVM shows the lowest variability.

To limit the computation cost once again, the Deep Learning models were only trained with a validation set without an additional cross-validation. The *early stopping* mechanism is based on the validation loss, with a patience set to 6 epochs. All the models used AdamW optimizer, an initial learning rate of 10^{-3} , a batch size of 64 elements and BCEWithLogitsLoss for the loss function.

Model	Best train ROC-AUC	Best valid ROC-AUC
MLP	0.9165	0.8340
Tab-Transformer-V1	0.8093	0.8042
Tab-Transformer-V2	0.8194	0.8136

Table 17: Deep Learning models training results

The MLP got the best ROC-AUC in validation, but the gap with its training performance suggests a bit of overfitting. These results must be interpreted carefully due to the absence of cross-validation and **limited size of the dataset**.

Note: The values highlighted in red represent the best results for each indicator.

4 Results and discussion

The final performances were evaluated on the test set in two steps.

First, they were assessed using the following metrics:

- **Accuracy**
- **Precision**
- **Recall**
- **F1-Score**
- **ROC-AUC**
- **PR-AUC**

In a second phase, confusion matrices were built in order to complete the metrics evaluation previously done. **These matrices allow to distinguish true positives (TP), true negatives (TN), false positives (FP) and false negatives (FN).**

4.1 Metrics evaluation

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC	PR-AUC
Logistic Regression	0.8428	0.8763	0.8333	0.8543	0.9269	0.9384
SVM	0.8750	0.8692	0.9118	0.8900	0.9316	0.9157
Random Forest	0.8696	0.8611	0.9118	0.8857	0.9559	0.9633
XGBoost	0.8859	0.8785	0.9216	0.8995	0.9443	0.9537
CatBoost	0.8696	0.8750	0.8922	0.8835	0.9266	0.9145
MLP	0.8261	0.8500	0.8333	0.8416	0.9206	0.9396
Tab-Transformer-V1	0.8152	0.7881	0.9118	0.8455	0.8835	0.8880
Tab-Transformer-V2	0.8261	0.8241	0.8725	0.8476	0.8923	0.8952

Table 18: Evaluation results

The results highlight two models in particular: XGBoost gets the best classification performances at the decision threshold used (0.5), while Random Forest dominates the discrimination metrics computed on all the thresholds. So, the choice of the model depends on what we want to prioritize for the project. The Deep Learning architectures do not outperform the best classical models and that can be explained especially by the limited size of the dataset. The detailed rankings are exposed in the [dedicated section](#).

4.2 Confusion matrices analysis

Model	TN	FP	FN	TP
Logistic Regression	64	18	11	91
SVM	68	14	9	93
Random Forest	67	15	9	93
XGBoost	69	13	8	94
CatBoost	74	8	19	89
MLP	67	15	17	85
Tab-Transformer-V1	57	23	16	86
Tab-Transformer-V2	63	19	13	89

Table 19: Confusion matrices results

The confusion matrices confirm the main trade-offs: XGBoost minimizes false negatives and detect the most bigger number of true positives while CatBoost minimizes false positives at the cost of more false positives. Deep Learning models commit more errors overall. Once again, the choice depends on the cost attributed to both false negatives and false positives.

5 Features importance

Features importance was analysed for the classical Machine Learning models to complete the correlation study and perhaps discover what are the most important features when we talk about coronary disease. This analysis is based on the **coefficients of logistic regression** and the **intern measurements of models based on trees**. SVM and Deep Learning models were excluded from this analysis, for they would required external methods such as *permutation importance* or *SHAP*.

A common synthesis of the different models results indicated that *oldpeak*, *chol*, *ca*, *age* and *thalch* appear among the most important features. The missing flag of *ca* contributed also to the predictions, probably due to the high number of missing values for this variable. **Although these results are consistent with some associations observed during the correlation analysis, they cannot be used to establish any certain causal relationship.**

6 Calibration

The calibration evaluates the reliability of the probabilities predicted in addition to the discrimination metrics. The **Platt method** was used and adjusted on the calibration dataset, then evaluated using metrics such as **Negative Log-Likelihood (NLL)**, **Brier Score** and **Expected Calibration Error (ECE)**.

Classification metrics and the confusion matrices were also examined in order to measure the effect of the new probabilistic threshold on the final predictions. However, both ROC-AUC and PR-AUC remain unchanged, as the sigmoid transformation preserves the relative ordering of the scores.

6.1 Calibration metrics evaluation

WITHOUT CALIBRATION

Model	NLL	Brier-Score	ECE
Logistic Regression	0.5217	0.1300	0.1248
SVM	0.4048	0.1060	0.0610
Random Forest	0.3095	0.0901	0.1052
XGBoost	0.3143	0.0932	0.0906
CatBoost	0.3488	0.1053	0.0718
MLP	0.3840	0.1189	0.0688
Tab-Transformer-V1	0.4484	0.1415	0.1238
Tab-Transformer-V2	0.4074	0.1279	0.0748

Table 20: Evaluation results (without calibration)

WITH CALIBRATION

Model	NLL	Brier-Score	ECE
Logistic Regression	0.4732	0.1321	0.1137
SVM	0.4048	0.1060	0.0924
Random Forest	0.3768	0.1074	0.1574
XGBoost	0.5721	0.1200	0.1186
CatBoost	0.5136	0.1182	0.1035
MLP	0.3841	0.1189	0.0688
Tab-Transformer-V1	0.4484	0.1415	0.1238
Tab-Transformer-V2	0.4074	0.1279	0.0748

Table 21: Evaluation results (with calibration)

For these three metrics, a little value is preferable. The calibration improved a bit NLL and ECE for the logistic regression, but degraded the results of models based on trees overall (especially XGBoost). Finally, the calibration effect is negligible on the Deep Learning models.

As a matter of fact, the Platt calibration (sigmoid) is not always benefic and several models already produced satisfactory probabilities before its application. The detailed rankings can be discovered in the [next section](#).

6.2 Confusion matrices analysis

Model	TN	FP	FN	TP
Logistic Regression	70	12	17	85
SVM	74	8	17	85
Random Forest	67	15	8	94
XGBoost	71	11	8	88
CatBoost	69	13	11	91
MLP	67	15	17	85
Tab-Transformer-V1	57	23	16	86
Tab-Transformer-V2	63	19	13	89

Table 22: Confusion matrices results (with calibration)

After calibration, Random Forest minimises false negatives, while SVM minimises false positives. The Deep Learning models predictions remain virtually unchanged. These results show that the calibration can shift the trade-off between the two error types, without neceserly improving the overall quality of the predictions.

7 Model rankings

DISCRIMINATION

ROC-AUC	PR-AUC
1 - Random Forest	1 - Random Forest
2 - XGBoost	2 - XGBoost
3 - SVM	3 - MLP
4 - Logistic Regression	4 - Logistic Regression
5 - CatBoost	5 - SVM
6 - MLP	6 - CatBoost
7 - Tab-Transformer-V2	7 - Tab-Transformer-V2
8 - Tab-Transformer-V1	8 - Tab-Transformer-V1

Table 23: Best discrimination ranking

CLASSIFICATION

Accuracy Rank	Precision Rank	Recall Rank	F1-Score Rank
1 - XGBoost	1 - XGBoost	1 - XGBoost	1 - XGBoost
2 - SVM	2 - Logistic Regression	2 - SVM	2 - SVM
3 - Random Forest	3 - CatBoost	2 - Random Forest	3 - Random Forest
3 - CatBoost	4 - SVM	2 - Tab-Transformer-V1	4 - CatBoost
4 - Logistic Regression	5 - Random Forest	3 - CatBoost	5 - Logistic Regression
5 - MLP	6 - MLP	4 - Tab-Transformer-V2	6 - Tab-Transformer-V2
6 - Tab-Transformer-V2	7 - Tab-Transformer-V2	5 - Logistic Regression	7 - Tab-Transformer-V1
7 - Tab-Transformer-V1	8 - Tab-Transformer-V1	5 - MLP	5 - MLP

Table 24: Best classification ranking (without calibration)

Accuracy Rank	Precision Rank	Recall Rank	F1-Score Rank
1 - Random Forest (+ 2)	1 - SVM (+ 2)	1 - XGBoost (-)	1 - XGBoost (-)
2 - XGBoost (- 1)	2 - XGBoost (- 1)	1 - Random Forest (+ 1)	2 - SVM (-)
2 - SVM (-)	3 - Logistic Reg (-)	2 - CatBoost (+ 1)	3 - Random Forest (-)
3 - CatBoost (-)	4 - CatBoost (- 1)	3 - Tab-Transformer-V2 (+ 1)	4 - CatBoost (-)
4 - Logistic Reg (-)	5 - Random Forest (-)	4 - Tab-Transformer-V1 (- 2)	5 - Logistic Reg (-)
4 - MLP (+ 1)	5 - MLP (+ 1)	5 - SVM (-3)	6 - Tab-Transformer-V2 (-)
4 - Tab-Transformer-V2 (+ 1)	6 - Tab-Transformer-V2 (+ 1)	5 - Logistic Reg (-)	7 - Tab-Transformer-V1 (-)
5 - Tab-Transformer-V1 (+ 1)	7 - Tab-Transformer-V1 (-)	5 - MLP (-)	5 - MLP (-)

Table 25: Best classification ranking (with calibration)

PROBABILITY QUALITY

NLL Rank	Brier-Score Rank	ECE Rank
1 - Random Forest	1 - Random Forest	1 - SVM
2 - XGBoost	2 - XGBoost	2 - MLP
3 - CatBoost	3 - CatBoost	3 - CatBoost
4 - MLP	4 - SVM	4 - Tab-Transformer-V2
5 - SVM	5 - MLP	5 - XGBoost
6 - Tab-Transformer-V2	6 - Tab-Transformer-V2	6 - Random Forest
7 - Tab-Transformer-V1	7 - Logistic Reg	7 - Tab-Transformer-V1
8 - Logistic Reg	8 - Tab-Transformer-V1	8 - Logistic Reg

Table 26: Best probability quality ranking (without calibration)

NLL Rank	Brier-Score Rank	ECE Rank
1 - Random Forest (-)	1 - SVM (+ 3)	1 - MLP (+ 1)
2 - MLP (+ 2)	2 - Random Forest (- 1)	2 - Tab-Transformer-V2 (+ 2)
3 - SVM (+ 2)	3 - CatBoost (-)	3 - SVM (- 2)
4 - Tab-Transformer-V2 (+ 2)	4 - MLP (+ 1)	4 - CatBoost (- 1)
5 - Tab-Transformer-V1 (+ 2)	5 - XGBoost (- 3)	5 - Logistic Reg (+ 3)
6 - Logistic Reg (+ 2)	6 - Tab-Transformer-V2 (-)	6 - XGBoost (- 1)
7 - CatBoost (- 4)	7 - Logistic Reg (-)	7 - Tab-Transformer-V1 (-)
8 - XGBoost (- 6)	8 - Tab-Transformer-V1 (-)	8 - Random Forest (- 2)

Table 27: Best probability quality ranking (with calibration)

8 Conclusion

This project compared several Machine Learning and Deep Learning approaches for detecting coronary disease from the UCI Heart Disease dataset. **The original multiclass target was transformed into a binary target**, and a reproducible pipeline was developed to handle missing values, encode categorical variables and standardise numerical features while preventing data leakage.

Overall, the classical Machine Learning models achieved the strongest results. XGBoost obtained the best threshold-based classification performance, with an Accuracy of 0.8859 and an F1-Score of 0.8995, while **Random Forest achieved the highest discrimination performance** with a ROC-AUC of 0.9559 and a PR-AUC of 0.9633. The Deep Learning models did not outperform the best classical approaches, suggesting that **their additional complexity was not beneficial for this relatively small tabular dataset**.

Platt calibration did not systematically improve the predicted probabilities. Its effect depended on the model and it notably degraded several metrics for the tree-based models, which were **already reasonably well calibrated**. These results highlight the importance of evaluating discrimination, classification performance and probability quality separately rather than selecting a model from a single metric.

The conclusions remain limited by the dataset size, the large number of missing values in several features and the use of a single test split. Further work could include external validation, repeated cross-validation, decision-threshold optimisation and additional interpretability methods (especially for Deep Learning models).

Within the scope of this project, XGBoost and Random Forest nevertheless provide the best overall trade-offs for coronary disease detection.

References

- [1] Andras Janosi, William Steinbrunn, Matthias Pfisterer, and Robert Detrano. Heart disease, 1988. Dataset downloaded from Kaggle.